Python for Data Analysis
Ali Bou Nassif

Python Libraries for Data Science

Many popular Python toolboxes/libraries:

- NumPy
- SciPy
- Pandas
- SciKit-Learn

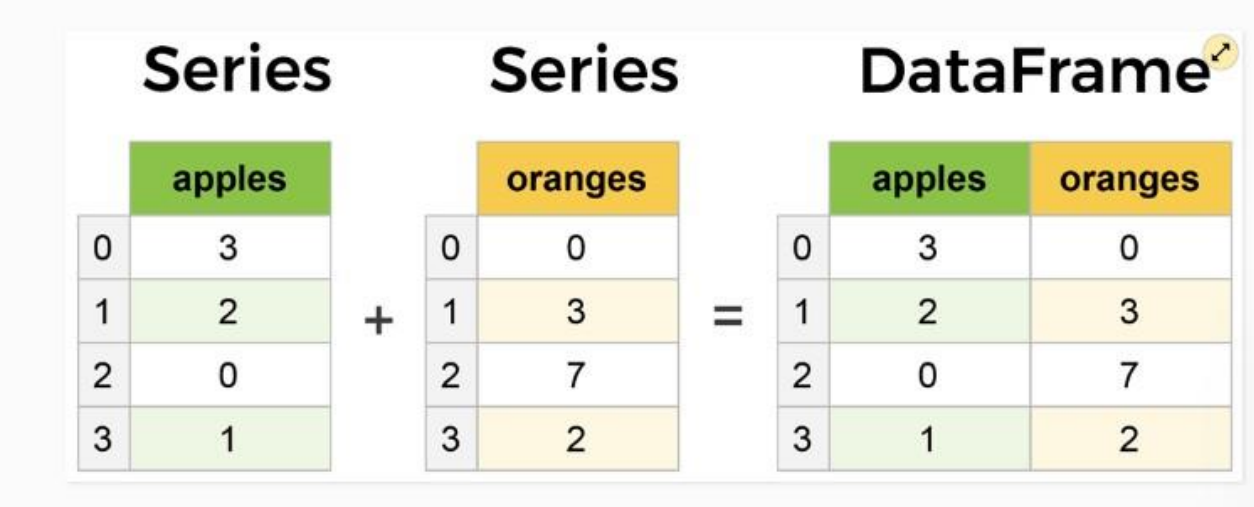
Visualization libraries

- matplotlib
- Seaborn

Pandas

The primary two components of pandas are the Series and DataFrame.

A Series is essentially a column, and a DataFrame is a multi-dimensional table made up of a collection of Series.



	Series		Series		DataFrame	
	apples		oranges		apples	oranges
0	3	+	0	=	3	0
1	2		3		2	3
2	0		7		0	7
3	1		2		1	2

Pandas

Pandas

In python we can import the library and use the functions inside it. We can also rename the library using the "as" command to make it easier to use in the code. The following line imports the pandas.

```
[1]  import pandas as pd # we imported pandas library and to make it simpler we will use as pd
```

The primary data structures in pandas are implemented as two classes:

- DataFrame, which you can imagine as a relational data table, with rows and named columns.
- Series, which is a single column. A DataFrame contains one or more Series and a name for each Series.

One way to create a Series is to construct a Series object.

```
[3]  pd.Series(['Sharjah', 'Ajman', 'Dubai'])
```

```
0    Sharjah
1      Ajman
2      Dubai
dtype: object
```

Pandas

DataFrame objects can be created by passing a dict mapping string column names to their respective Series. If the Series don't match in length, missing values are filled with special NA/NaN values.

```
[4]  city_names = pd.Series(['Sharjah', 'Ajaman', 'Dubai'])
     population = pd.Series([1275000, 490035, 3331000])

     pd.DataFrame({ 'City name': city_names, 'Population': population })
```

	City name	Population
0	Sharjah	1275000
1	Ajaman	490035
2	Dubai	3331000

Pandas

```
# Create and populate a 5x2 NumPy array.
import numpy as np
my_data = np.array([[0, 3], [10, 7], [20, 9], [30, 14], [40, 15]])

# Create a Python list that holds the names of the two columns.
my_column_names = ['temperature', 'activity']

# Create a DataFrame.
my_dataframe = pd.DataFrame(data=my_data, columns=my_column_names)

# Print the entire DataFrame
print(my_dataframe)
```

```
   temperature  activity
0            0         3
1           10         7
2           20         9
3           30        14
4           40        15
```

Hands-on exercises

✓ Write a code to create the following dataframe

	A	B	G
0	1	2	m
1	3	4	f

Reading data using pandas

```
import pandas as pd
```

```
#Read csv file
```

```
# upload csv file to Google Colab
from google.colab import files
uploaded = files.upload()
df = pd.read_csv('Salaries.csv')
```

Data Frames attributes

Python objects have *attributes* and *methods*. Try them

df.attribute	description
dtypes	list the types of the columns
columns	list the column names
axes	list the row labels and column names
ndim	number of dimensions
size	number of elements
shape	return a tuple representing the dimensionality
values	numpy representation of the data

Data Frames methods

df.method()	description
head(n), tail(n)	first/last n rows
describe()	generate descriptive statistics (for numeric columns only)
max(), min()	return max/min values for all numeric columns
mean(numeric_only = True), median(numeric_only = True) mode	return mean/median values for all numeric columns Mode returns the value(s) that appear most often in the data.
std(numeric_only = True), Var(numeric_only = True)	standard deviation, variance
sample(n)	returns a random sample of the data frame
dropna()	drop all the records with missing values
Sem (numeric_only = True)	Standard error of mean
Skew (numeric_only = True)	Sample skewness
Kurt (numeric_only = True)	kurtosis

Difference between Attributes and Methods

Feature	Attribute	Method
What it is	A variable stored inside an object	A function defined inside an object
How to access	`object.attribute`	`object.method()`
Example	`df.columns`	`df.describe()`
Parentheses	❌ No	✅ Yes
Purpose	Store or show data	Perform an operation

Exploring data frames

upload the file Salaries.csv

```
#Getting some info about your data

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 78 entries, 0 to 77
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   rank        78 non-null     object
 1   discipline  78 non-null     object
 2   phd         78 non-null     int64
 3   service     78 non-null     int64
 4   sex         78 non-null     object
 5   salary      78 non-null     int64
dtypes: int64(3), object(3)
memory usage: 3.8+ KB
```

Exploring data frames

upload the file Salaries.csv

```
#Getting the shape of your data (number of rows and number of columns)

df.shape
```

```
(78, 6)
```

Exploring data frames

```
upload the file Salaries.csv
```

```
#List first 5 records
#df.head(n); default value 5

df.head()
```

	rank	discipline	phd	service	sex	salary
0	Prof	B	56	49	Male	186960
1	Prof	A	12	6	Male	93000
2	Prof	A	23	20	Male	110515
3	Prof	A	40	31	Male	131205
4	Prof	B	20	18	Male	104800



Hands-on exercises

- ✓ Try to read the first 10, 20, 50 records;
- ✓ Can you guess how to view the last few records; Hint: what is opposite to head?

Data Frame data types

Pandas Type	Native Python Type	Description
object	string	The most general dtype. Will be assigned to your column if column has mixed types (numbers and strings).
int64	int	Numeric characters. 64 refers to the memory allocated to hold this character.
float64	float	Numeric characters with decimals. If a column contains numbers and NaNs(see below), pandas will default to float64, in case your missing value has a decimal.
datetime64, timedelta[ns]	N/A (but see the datetime module in Python's standard library)	Values meant to hold time data. Look into these for time series experiments.

Data Frame data types

```
#Check a particular column type
df['salary'].dtype
```

```
dtype('int64')
```

```
#Check types for all the columns
df.dtypes
```

```
rank              object
discipline        object
phd               int64
service           int64
sex               object
salary            int64
dtype: object
```

Examples

For the whole set (it will be checked for each column)

```
[57]  df.mode() # returns the most frequent value for ea
```

	rank	discipline	phd	service	sex	salary
0	Prof	B	4.0	3.0	Female	92000.0
1	NaN	NaN	NaN	NaN	Male	NaN

For column discipline only

```
display(
    df.discipline.mode(),
)
```

	discipline
0	B

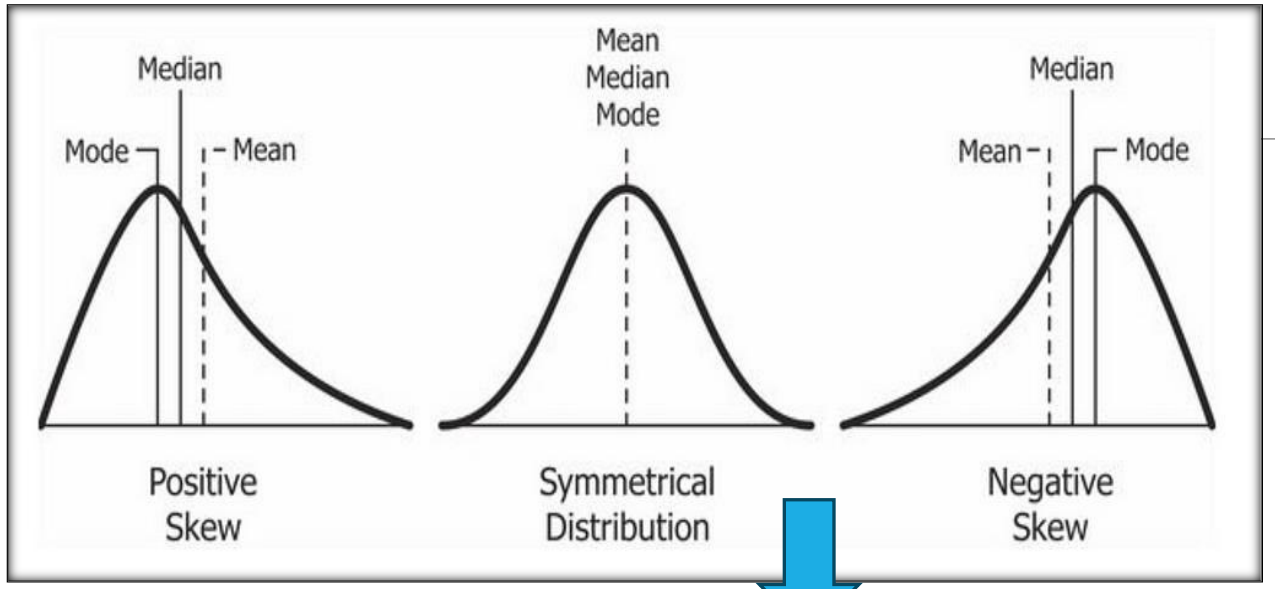
dtype: object

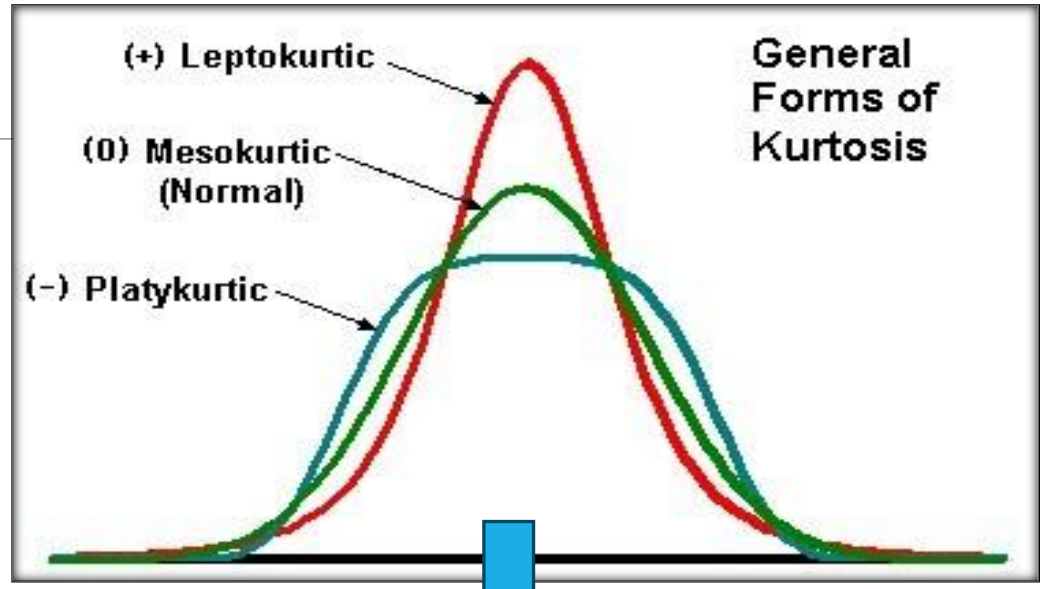
```
df['discipline'].mode()
```

	discipline
0	B

dtype: object

Examples






```
df.skew(numeric_only=True) # returns the skewness for numeric cols
```

	0
phd	0.634366
service	0.913750
salary	0.452103

dtype: float64

```
df.kurt(numeric_only=True) # returns the kurtosis for numeric cols
                           # if 0 - normal bell shape
                           # if > 0 - sharp peak
                           # if < 0 - light peak
```

	0
phd	0.042504
service	0.608981
salary	-0.401713

dtype: float64



Hands-on exercises

- ✓ Give the summary for the numeric columns in the dataset Salaries
- ✓ Calculate standard deviation for all numeric columns;
- ✓ What are the mean values of the first 50 records in the dataset? ***Hint:*** use head() method to subset the first 50 records and then calculate the mean

Selecting a column in a Data Frame

Method 1: **Subset the data frame using column name:**

df['sex']

***Method 2*:** **Use the column name as an attribute:**

df.sex

Note: **preferred to use method 1**

Handling duplicates

Create duplicate data by using concat method

```
dataframe_1 = pd.DataFrame(data=data_1, columns=column_names)
print(dataframe_1)
print()

dataframe_2 = pd.DataFrame({'G': data_2})
print(dataframe_2)
print()

dataframe_f = pd.concat([dataframe_1,dataframe_2] , axis=1)
print(dataframe_f)
print()
print(dataframe_f.shape)
print()
```

```
   A  B
0  1  2
1  3  4

   G
0  m
1  f

   A  B  G
0  1  2  m
1  3  4  f

(2, 3)
```

Handling duplicates

Drop duplicates

Handling duplicates

```
[29]  # Create duplicate using append
      df_temp = pd.concat([df, df], axis=0) # I want to duplicate rows not cols

      print(df_temp.shape, '\n')

      df_temp = df_temp.drop_duplicates()

      print(df_temp.shape)
```

```
(156, 6)

(78, 6)
```

Data Frame: filtering

To subset the data we can apply Boolean indexing. This indexing is commonly known as a filter. For example if we want to subset the rows in which the salary value is greater than $120K:

```
df_sub = df[ df['salary'] > 120000 ]
```

Any Boolean operator can be used to subset the data:

> greater; >= greater or equal;

< less; <= less or equal;

== equal; != not equal;

```
#Select only those rows that contain female professors:
df_f = df[ df['sex'] == 'Female' ]
```

Data Frames: Slicing

There are a number of ways to subset the Data Frame:

- one or more columns
- one or more rows
- a subset of rows and columns

Rows and columns can be selected by their position or label

Data Frames: Slicing

When selecting one column, it is possible to use single set of brackets, but the resulting object will be a Series (not a DataFrame):

```
#Select column salary:
df['salary']
```

When we need to select more than one column and/or make the output to be a DataFrame, we should use double brackets:

```
#Select column salary:
df[['rank','salary']]
```

Data Frames: Selecting rows

If we need to select a range of rows, we can specify the range using ":"

```
#Select rows by their position:
df[10:20]
```

Notice that the first row has a position 0, and the last value in the range is omitted: So for 0:10 range the first 10 rows are returned with the positions starting with 0 and ending with 9

Data Frames *groupby* method

Using "group by" method we can:

- Split the data into groups based on some criteria
- Calculate statistics (or apply a function) to each group

```
#Group data using rank
df_rank = df.groupby(['rank'])
```

```
#Calculate mean value for each numeric column per each group
df_rank.mean(numeric_only=True)
```

rank	phd	service	salary
AssocProf	15.076923	11.307692	91786.230769
AsstProf	5.052632	2.210526	81362.789474
Prof	27.065217	21.413043	123624.804348

Data Frames *groupby* method

Once groupby object is create we can calculate various statistics for each group:

```
#Calculate mean salary for each professor rank:
df.groupby('rank')[['salary']].mean()
```

	salary
rank	
AssocProf	91786.230769
AsstProf	81362.789474
Prof	123624.804348

Note: If single brackets are used to specify the column (e.g. salary), then the output is Pandas Series object. When double brackets are used the output is a Data Frame



Hands-on exercises

- ✓ Calculate the basic statistics for the *salary* column;
- ✓ Find how many values in the *salary* column (use *count* method);
- ✓ Calculate the average salary;

Data Frames: method loc

If we need to select a range of rows, using their labels we can use method loc:

```
#Select rows by their labels:
df_sub.loc[10:20,['rank','sex','salary']]
```

	rank	sex	salary
10	Prof	Male	128250
11	Prof	Male	134778
13	Prof	Male	162200
14	Prof	Male	153750
15	Prof	Male	150480
19	Prof	Male	150500

Data Frames: method iloc

If we need to select a range of rows and/or columns, using their positions we can use method iloc:

```
#Select rows by their labels:
df_sub.iloc[10:20,[0, 3, 4, 5]]
```

	rank	service	sex	salary
26	Prof	19	Male	148750
27	Prof	43	Male	155865
29	Prof	20	Male	123683
31	Prof	21	Male	155750
35	Prof	23	Male	126933
36	Prof	45	Male	146856
39	Prof	18	Female	129000
40	Prof	36	Female	137000
44	Prof	19	Female	151768
45	Prof	25	Female	140096

Data Frames: method iloc (summary)

```
df.iloc[0]  # First row of a data frame
df.iloc[i]  #(i+1)th row
df.iloc[-1] # Last row
```

```
df.iloc[:, 0]  # First column
df.iloc[:, -1] # Last column
```

```
df.iloc[0:7]        #First 7 rows
df.iloc[:, 0:2]     #First 2 columns
df.iloc[1:3, 0:2]  #Second through third rows and first 2 columns
df.iloc[[0,5], [1,3]]  #1st and 6th rows and 2nd and 4th columns
```

Data Frames: Sorting

We can sort the data by a value in the column. By default the sorting will occur in ascending order and a new data frame is return.

In [ ]:
```
# Create a new data frame from the original sorted by the column Salary
df_sorted = df.sort_values( by ='service', ascending=True)
df_sorted.head()
```

Out[ ]:

	rank	discipline	phd	service	sex	salary
55	AsstProf	A	2	0	Female	72500
23	AsstProf	A	2	0	Male	85000
43	AsstProf	B	5	0	Female	77000
17	AsstProf	B	4	0	Male	92000
12	AsstProf	B	1	0	Male	88000

Data Frames: Sorting

We can sort the data using 2 or more columns:

```
df_sorted = df.sort_values( by =['service', 'salary'], ascending = [True, False])
df_sorted.head(10)
```

	rank	discipline	phd	service	sex	salary
52	Prof	A	12	0	Female	105000
17	AsstProf	B	4	0	Male	92000
12	AsstProf	B	1	0	Male	88000
23	AsstProf	A	2	0	Male	85000
43	AsstProf	B	5	0	Female	77000
55	AsstProf	A	2	0	Female	72500
57	AsstProf	A	3	1	Female	72500
28	AsstProf	B	7	2	Male	91300
42	AsstProf	B	4	2	Female	80225
68	AsstProf	A	4	2	Female	77500



Hands-on exercises

✓ *Regarding the previous slide, sort the data in descending order*

Data Cleansing

Remove irrelevant columns based on experience

ID	Gender	age	applicant salary (k)	spouse sarary (K)	total salary (K)	smoker	nationality	maximum mortgage [out]
1	m	25	22	12	34	Y	Egypt	800 K
2	m	60	20	28	48	Y	Japan	150 K
3	f	33	30	20	50	N	UAE	700 K
4	f	45	24	34	58	N	Jordan	540 K
5	f	40	33	25	58	N	UAE	980 K
6	m	32	41	14	55	N	Jordan	650 K

Data Cleansing

Remove columns that do not correlate to the output based on statistics (later in next lab)

- Filters
- Wrappers

Data Cleansing

Rows with missing values

- Drop these rows `df.dropna()`
- Replace missing values with mean / median (later in next lab)

Data Cleansing

Outliers

- Mild outliers
- Extreme outliers

Missing Values

Missing values are marked as NaN

```
# Read a dataset with missing values
```

```
# upload the dataset flights.csv
```

```
# Select the rows that have at least one missing value
flights = pd.read_csv('flights.csv')
flights[flights.isnull().any(axis=1)].head()
```

	year	month	day	dep_time	dep_delay	arr_time	arr_delay	carrier	tailnum	flight	origin	dest	air_time	distance	hour	minute
330	2013	1	1	1807.0	29.0	2251.0	NaN	UA	N31412	1228	EWR	SAN	NaN	2425	18.0	7.0
403	2013	1	1	NaN	NaN	NaN	NaN	AA	N3EHAA	791	LGA	DFW	NaN	1389	NaN	NaN
404	2013	1	1	NaN	NaN	NaN	NaN	AA	N3EVAA	1925	LGA	MIA	NaN	1096	NaN	NaN
855	2013	1	2	2145.0	16.0	NaN	NaN	UA	N12221	1299	EWR	RSW	NaN	1068	21.0	45.0
858	2013	1	2	NaN	NaN	NaN	NaN	AA	NaN	133	JFK	LAX	NaN	2475	NaN	NaN

Missing Values

```
flights.isnull().sum()

year            0
month           0
day             0
dep_time     2336
dep_delay    2336
arr_time     2479
arr_delay    2827
carrier         0
tailnum      1433
flight          0
origin          0
dest            0
air_time     2827
distance        0
hour         2336
minute       2336
dtype: int64
```

Example

	flight	delay	price
0	AA1	10.0	300
1	BB2	NaN	450
2	CC3	20.0	NaN

After `flights.isnull()`:

	flight	delay	price
0	False	False	False
1	False	True	False
2	False	False	True



So we get:

0 False
1 True
2 True

any(axis=1)
This checks each row (because axis=1 means across columns).
If any value in that row is True (meaning missing), the result for that row is True.

Missing Values

There are a number of methods to deal with missing values in the data frame:

df.method()	description
dropna()	Drop missing observations
dropna(how='all')	Drop observations where all cells is NA
dropna(axis=1, how='all')	Drop column if all the values are missing
dropna(thresh = 5)	Drop rows that contain less than 5 non-missing values
fillna(0)	Replace missing values with zeros
isnull()	returns True if the value is missing
notnull()	Returns True for non-missing values

Dropping rows with missing Values

```
flights=flights.dropna()
flights.head()
```

	year	month	day	dep_time	dep_delay	arr_time	arr_delay	carrier	tailnum	flight	origin	dest	air_time	distance	hour	minute
0	2013	1	1	517.0	2.0	830.0	11.0	UA	N14228	1545	EWR	IAH	227.0	1400	5.0	17.0
1	2013	1	1	533.0	4.0	850.0	20.0	UA	N24211	1714	LGA	IAH	227.0	1416	5.0	33.0
2	2013	1	1	542.0	2.0	923.0	33.0	AA	N619AA	1141	JFK	MIA	160.0	1089	5.0	42.0
3	2013	1	1	554.0	-6.0	812.0	-25.0	DL	N668DN	461	LGA	ATL	116.0	762	5.0	54.0
4	2013	1	1	554.0	-4.0	740.0	12.0	UA	N39463	1696	EWR	ORD	150.0	719	5.0	54.0

Dropping rows with missing Values

```
flights=flights.dropna()
flights.isnull().sum()
```

```
year          0
month         0
day           0
dep_time      0
dep_delay     0
arr_time      0
arr_delay     0
carrier       0
tailnum       0
flight        0
origin        0
dest          0
air_time      0
distance      0
hour          0
minute        0
dtype: int64
```

Testing

Aggregation Functions in Pandas

Aggregation - computing a summary statistic about each group, i.e.

- compute group sums or means
- compute group sizes/counts

Common aggregation functions:

min, max
count, sum
mean, median, mode
std, var

Aggregation Functions in Pandas

agg() method are useful when multiple statistics are computed per column:

```
flights[['dep_delay','arr_delay']].agg(['min','mean','max'])
```

	dep_delay	arr_delay
min	-16.000000	-62.000000
mean	9.384302	2.298675
max	351.000000	389.000000

Practice

Task 1 :

• Create a 3x4 (3 rows x 4 columns) pandas DataFrame in which the columns are named Mustafa, Sara, Khalid, and Muna. Populate each of the 12 cells in the DataFrame with a random integer between 0 and 100, inclusive.

• Output the following:
  - the entire DataFrame
  - the values of row #1 with the header (print as dataframe)

• Create a fifth column named Ahmed, which is populated with the sums of columns Khalid' and Muna'.

Create a sixth column named Grade, which indicate if the student pass or not. (pass means grade is above 60, True means pass).

Data Frames: Selecting rows

If we need to select a range of rows, we can specify the range using ":"

```
#Select rows by their position:
df[10:20]
```

	rank	discipline	phd	service	sex	salary
10	Prof	B	39	33	Male	128250
11	Prof	B	23	23	Male	134778
12	AsstProf	B	1	0	Male	88000
13	Prof	B	35	33	Male	162200
14	Prof	B	25	19	Male	153750
15	Prof	B	17	3	Male	150480
16	AsstProf	B	8	3	Male	75044
17	AsstProf	B	4	0	Male	92000
18	Prof	A	19	7	Male	107300
19	Prof	A	29	27	Male	150500

Data Frames: method loc

If we need to select a range of rows, using their labels we can use method loc. Pandas **includes both endpoints** (start **and** stop labels) in the result.

```
#Select rows by their labels:
df.loc[10:20]
```

	rank	discipline	phd	service	sex	salary
10	Prof	B	39	33	Male	128250
11	Prof	B	23	23	Male	134778
12	AsstProf	B	1	0	Male	88000
13	Prof	B	35	33	Male	162200
14	Prof	B	25	19	Male	153750
15	Prof	B	17	3	Male	150480
16	AsstProf	B	8	3	Male	75044
17	AsstProf	B	4	0	Male	92000
18	Prof	A	19	7	Male	107300
19	Prof	A	29	27	Male	150500
20	AsstProf	B	4	4	Male	92000

Data Frames: method loc

If we need to select a range of rows, using their labels we can use method loc:

```
#Select rows by their labels:
df_sub.loc[10:20,['rank','sex','salary']]
```

	rank	sex	salary
10	Prof	Male	128250
11	Prof	Male	134778
13	Prof	Male	162200
14	Prof	Male	153750
15	Prof	Male	150480
19	Prof	Male	150500

Data Frames: method iloc

If we need to select a range of rows and/or columns, using their positions we can use method iloc(Integer-location based indexing):

```
#Select rows by their labels:
df_sub.iloc[10:20,[0, 3, 4, 5]]
```

	rank	service	sex	salary
26	Prof	19	Male	148750
27	Prof	43	Male	155865
29	Prof	20	Male	123683
31	Prof	21	Male	155750
35	Prof	23	Male	126933
36	Prof	45	Male	146856
39	Prof	18	Female	129000
40	Prof	36	Female	137000
44	Prof	19	Female	151768
45	Prof	25	Female	140096

Dropping a column by the name of the column

```
x = df.drop(['rank'], axis=1)
x.head(3)
```

	discipline	phd	service	sex	salary
0	B	56	49	Male	186960
1	A	12	6	Male	93000
2	A	23	20	Male	110515

Why?

```
x = df.drop(['rank', 'phd'], axis=1)#x = df.drop(columns=['rank', 'phd'], axis=1)
x.head(3)
```

	discipline	service	sex	salary
0	B	49	Male	186960
1	A	6	Male	93000
2	A	20	Male	110515

Dropping a column by the index of the column

```
x = df.drop(df.columns[[0]], axis=1)
x.head(3)
```

	discipline	phd	service	sex	salary
0	B	56	49	Male	186960
1	A	12	6	Male	93000
2	A	23	20	Male	110515

```
x = df.drop(df.columns[[0,2]], axis=1)
x.head(3)
```

	discipline	service	sex	salary
0	B	49	Male	186960
1	A	6	Male	93000
2	A	20	Male	110515